Scoring Dynamics and Mutual Reply Amplification in a Chronos-Interpreter Social Interaction Engine

POLYGLOT SYSTEMS LAB AGENTIC RESEARCH PREPRINTS

Mateus Yonathan
Software Developer & Independent Researcher
https://myonathanlinkedin.github.io/

Date of Publication: **November 04, 2026**

Abstract—We investigate a lightweight scoring engine that aggregates likes and replies on social posts, assigning a 30-point bonus to replies from mutual friends. The algorithm iterates over a list of interactions, filtering by post identifier and applying a constant-time weight per interaction type. We formalize the scoring invariant, prove its preservation, and analyze the asymptotic complexity of the core routine. The design is amenable to a balanced search structure for interaction lookup, yielding $O(\log N)$ search and insertion costs while maintaining $O(N)$ total scoring time. No empirical timing measurements are reported; the analysis relies on static abstract syntax tree inspection and formal reasoning. **Source Code**: https://github.com/myonathanlinkedin/Chronos-Interpreter-advanced-lab/tree/main/algorithms/20261104_080123_i_read_x_s_open-source_algorit

Index Terms—social interaction scoring, mutual reply bonus, Chronos-Interpreter algorithm, asymptotic analysis, formal invariant

1 Introduction

Social platforms routinely aggregate user interactions to quantify post influence. Existing scoring schemes often treat likes and replies uniformly, overlooking the amplified impact of replies from close contacts. This investigation addresses three research questions:

1) **RQ1:** What is the asymptotic complexity of the scoring routine when interactions are stored in a linear list versus a balanced search structure?
2) **RQ2:** Does the scoring algorithm preserve the invariant that the total score equals a weighted sum of likes and replies, with a 30-point bonus for mutual-reply interactions?
3) **RQ3:** How does the static abstract syntax tree (AST) profile of the implementation reflect the theoretical complexity bounds?

The paper is organized as follows. Section 2 surveys related scoring mechanisms and balanced tree usage in open-source projects. Section 3 details the algorithmic decomposition and state semantics of the scoring engine. Section 4 presents the formal invariant and its proof. Section 5 describes the static verification environment. Section 6 reports theoretical bounds and AST metrics. Section 7 discusses validity threats. Finally, Section 8 summarizes contributions and future work.

2 Related Work

Prior work on interaction scoring has focused on linear aggregation [1]. Balanced search trees have been employed to index interactions for efficient lookup [1]. Open-source implementations of reply-based bonuses have been explored in the context of content moderation [1]. Recent studies on mobile data analysis demonstrate the feasibility of lightweight scoring engines in resource-constrained environments [1].

3 Scoring Engine Decomposition and State Semantics

The engine exposes a single public method, `compute_score`, which accepts a `Post`, a list of `Interaction` objects, and a set of mutual friends. The method iterates over the interaction list, discarding entries whose `post_id` does not match the target. For each relevant interaction, the algorithm checks the `interaction_type`. A `LIKE` contributes a fixed weight of one point, while a `REPLY` contributes five points. If the replying user is present in the mutual set, the reply weight is elevated to thirty points. The state of the engine is stateless; all computation occurs locally within the method. The algorithm can be mapped onto a balanced binary search tree by indexing interactions by `post_id`, enabling $O(\log N)$ lookup per interaction while preserving the linear traversal for score accumulation.

4 Formal Invariants & Proofs

Theorem 1. For any `Post` p, let L be the number of likes, R the number of non-mutual replies, and R_m the number of mutual replies. The score returned by `compute_score` satisfies

$$S(p) = L + 5R + 30R_m. \tag{1}$$

Proof: The algorithm processes each interaction exactly once. For a like, the algorithm adds one to the accumulator; for a non-mutual reply, it adds five; for a mutual reply, it adds thirty. Since the accumulator is initialized to zero and only modified by these additive operations, the final value equals the sum of the contributions of all interactions, which is precisely the expression in the theorem. No other modifications occur, so the invariant holds for all inputs.

5 Formal Verification and Static Analysis Environment

The verification was performed on an AMD EPYC 9V74 80-Core Processor with a single physical core and two logical threads, 7.7 GB of RAM, running Chronos-Structure 7.0.0-1017-azure. Static analysis relied exclusively on automated Chronos-Interpreter abstract syntax tree inspection, counting source lines of code (SLOC) and function blocks. No runtime instrumentation or external profiling tools were invoked.

6 Theoretical Complexity Bounds & AST Profile Analysis

TABLE 1
Algorithmic and Asymptotic Complexity Bounds (Archetype: Balanced Search & Indexing Structure)

Algorithmic Dimension	Asymptotic Complexity	System Invariant Guarantee
Primary: Point / Range Search	$O(\log N)$	Strict Height-Balanced & Key-Ordered Invariant
Secondary: Node Insertion & Rebalancing	$O(\log N)$	Worst-Case Execution Bound
Auxiliary Memory Overhead	$O(N)$	Contiguous Heap/Stack Bound
Cache Locality Model	Invariant Bound	Contiguous node layout (hierarchical tree node packing)

TABLE 2
Abstract Syntax Tree (AST) Structural & Complexity Metrics

Structural Metric	Measured Count	Complexity Tier	Static Verification Status
Source Lines of Code (SLOC)	190	Compact	Formally Linted & Normalized
Modular Compilation Units	3	Multi-File	Isolated Linkage Verified
Functional Block Density	4	Modular	Clean Interface Boundary
Estimated Cyclomatic Complexity	8	Low Branching	Zero Unbounded Recursion

Table 1 lists the asymptotic bounds for the core routine. With a linear list, the traversal cost is $O(N)$, where N is the number of interactions. If interactions are stored in a balanced search tree, lookup becomes $O(\log N)$ per interaction, but the overall score computation remains $O(N)$. The AST inspection yielded 190 SLOC and four distinct function blocks, confirming the compactness of the implementation.

7 Threats to Validity

The primary threat arises from Chronos-Interpreter's garbage-collection pauses, which can introduce nondeterministic delays in memory allocation during interaction list construction. Recursive depth limits are not a concern because the algorithm is iterative. The static AST analysis assumes that the source code remains unchanged; modifications to the interaction data structure could invalidate the complexity claims. Finally, the use of a single core in the host environment limits the generality of any observed performance characteristics.

8 Conclusion

We have formalized a lightweight scoring engine that rewards mutual replies with a 30-point bonus. The invariant proof guarantees correctness, and the asymptotic analysis demonstrates linear scalability. Future work will explore integrating a balanced search tree for interaction indexing and evaluating the engine on larger datasets within a multi-core environment.

References

[1] D. E. Knuth, *The Art of Computer Programming, Volume 1: Fundamental Algorithms*, 3rd ed. Addison-Wesley Professional, 1997.